```
# GET /articles
# GET /articles.json
def index
  @articles = Article.find_recent
  respond_to do |format|
    format.html # index.html.erb
    format.json { render json: @articles }
end 13: end

(byebug)
(byebug) next
[4, 13] in /PathToProject/app/controllers/articles_controller.rb
end 13: end

(byebug)


(byebug) step
[49, 58] in /PathToGems/activesupport-
5.1.0/lib/active_support/core_ext/numeric/time.rb
49:

   50:   # Returns a Duration instance matching the number of weeks
provided.
   51:   #
   52:   #   2.weeks # => 14 days
53: => 54:

def weeks
  ActiveSupport::Duration.weeks(self)
55:   end
56:   alias :week :weeks
57:
58:   # Returns a Duration instance matching the number of
```

If we use next, we won't go deep inside method calls. Instead, byebug will go to the next line within the same context. In this case, it is the last line of the current method, so byebug will return to the next line of the caller method.

If we use step in the same situation, byebug will literally go to the next Ruby instruction to be executed -- in this case, Active Support's week method.